

Projeto Final

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Projetos

Este projeto é estritamente individual. Seleciona **um** dos seguintes projetos.

O repositório deve conter todos os scripts relevantes, ficheiros de configuração e um ficheiro README.md com instruções detalhadas sobre como fazer o deploy do projeto.

Este é o **projeto final**, desenhado para integrar as diversas competências adquiridas ao longo do semestre (Shell Scripting, Docker, Python, Análise de Dados e Tecnologias Web).

Este é um projeto com a duração de três semanas (prazo limite **22/06/2026**). Tens até ao final desta semana para notificar o teu professor (via e-mail) sobre o tema escolhido (a lista de temas pode ser consultada [aqui](#)).

Não te esqueças de contactar o teu professor em caso de dúvidas. Instruções adicionais podem ser adicionadas.

Temas

1. Plataforma de Monitorização IoT Empresarial com Backups e Caching Automáticos

- **Descrição:** Implementa um pipeline de monitorização IoT industrial. Múltiplos contentores de simulação de sensores (scripts em Python) publicam telemetria ambiental sintética (temperatura, humidade, vibração) via **MQTT**. Um *broker* **Mosquitto** encaminha esta telemetria. Um serviço de recolha **FastAPI** subscreve os tópicos MQTT, valida os dados recebidos e armazena-os numa base de dados **PostgreSQL**. O **Grafana** é implementado para visualizar as métricas em tempo real. Um contentor **Caddy** (ou Nginx) serve como proxy inverso de cache

em frente ao painel do Grafana. Um **script Bash de backup** corre periodicamente via cron dentro de um contentor de backup dedicado para exportar a base de dados PostgreSQL, comprimi-la num ficheiro `.tar.gz` com data/hora e rodar os arquivos antigos (mantendo apenas os 7 mais recentes).

- **Competências Chave:** Docker Compose (rede bridge multi-contentor), MQTT (Mosquitto), Persistência SQL (PostgreSQL com volumes persistentes), Proxy Inverso e Caching (Caddy/Nginx), Scripting Bash e Automação (cron, tar, rotação de ficheiros), Grafana.

2. Quadro de Tarefas Colaborativo com Ambientes de Desenvolvimento Customizados e Análítica de Produtividade

- **Descrição:** Cria um quadro Kanban colaborativo em tempo real. A aplicação web permite que múltiplos utilizadores gire tarefas, sincronizando os movimentos dos cartões entre clientes instantaneamente via **WebSockets**. O backend usa **Redis Pub/Sub** para gerir a concorrência na difusão de mensagens e armazena o estado persistente das tarefas numa base de dados **PostgreSQL**. Um contentor *offline* separado e personalizado executa um **script Python** utilizando **Pandas** ou **Polars** para ler a base de dados periodicamente, calcular métricas chave (como o tempo médio de ciclo por tarefa e distribuição de cargas de trabalho) e guardar um relatório analítico resumido. Um **script daemon Bash** corre no host (ou num contentor privilegiado) para monitorizar o tamanho do diretório de armazenamento, registar o uso de recursos do contentor de base de dados e registar batimentos de atividade (*heartbeats*) num log de sistema. Os programadores devem usar uma configuração de **Dev Container** (Dockerfile configurando ferramentas de depuração e linters) para o desenvolvimento local.
- **Competências Chave:** WebSockets, Redis Pub/Sub, PostgreSQL, Dev Containers (Dockerfile), Análítica de Dados em Python (Pandas/Polars), Scripting Bash Daemon (monitorização de integridade e disco), Docker Compose.

3. Centro de Análítica Ambiental Geo-Distribuído

- **Descrição:** Cria um painel geográfico que visualiza métricas ambientais. Um **serviço Python** consulta periodicamente uma **API REST externa** (ex: OpenWeatherMap, OpenAQ ou uma API pública de qualidade do ar) para recolher métricas meteorológicas ou ambientais em 5 cidades pré-selecionadas, persistindo os dados brutos numa base de dados. Um **script Python** de análise separado utilizando **Pandas** corre para detetar valores atípicos (*outliers*), calcular desvios padrão

(volatilidade) e exportar dados JSON estruturados para um volume partilhado. Um servidor web **Nginx** aloja um portal estático com um **mapa interativo Leaflet.js** que lê os dados JSON e exibe marcadores coloridos com base na gravidade da qualidade do ar ou do clima. Um **script Bash** corre com um temporizador para verificar a disponibilidade da API externa (usando `curl`), inspecionar os limites de taxa REST (*rate-limits*) e anexar logs de estado.

- **Competências Chave:** APIs REST Externas, Análise de Dados (Pandas), Visualização de Mapas (Leaflet.js), Servidor Web Nginx, Automação Bash (verificação de API com `curl`), Docker Compose.

4. Cofre de Documentos e Média Seguro e Multi-Utilizador

- **Descrição:** Implementa uma biblioteca de documentos segura que automatiza a recolha de metadados e a proteção criptográfica. Um serviço web **FastAPI** gere uploads autenticados de ficheiros (documentos ou média) e consulta automaticamente uma **API REST externa** (ex: TMDb ou OpenLibrary) para extrair metadados ricos com base no título, guardando os metadados numa base de dados. Quando um novo ficheiro é carregado, um **script Bash** em segundo plano atuando como observador de diretório (*directory watcher*) deteta o ficheiro, verifica a sua integridade (hashing SHA256), encripta-o usando **GnuPG** ou **OpenSSL** com uma frase-chave/chave segura e elimina o original não encriptado. O sistema deve impor permissões estritas de **ficheiros Linux** (`chmod/chown` no volume) para que apenas o agente de encriptação possa ler os ficheiros brutos. Uma interface web exibe o catálogo de metadados e permite que utilizadores autorizados solicitem, desencriptem e transmitam ficheiros dinamicamente.
- **Competências Chave:** Conceitos de Segurança (encriptação GnuPG/OpenSSL, hash SHA256), Permissões do Sistema Linux (`chmod/chown`), Uploads FastAPI, Persistência em Base de Dados, Extração de APIs REST Externas, Monitorização de Diretórios em Bash.

5. Pipeline de CI/CD de Relatórios Markdown Baseado em Base de Dados

- **Descrição:** Desenha um pipeline automatizado de Integração Contínua que compila relatórios analíticos a partir de registos da base de dados. Uma aplicação multi-contentor armazena métricas de sistema ou registos de e-commerce numa base de dados **PostgreSQL**. Um **script Bash** monitoriza uma vista de dados ou pasta de logs do PostgreSQL para atualizações. Quando uma alteração é detetada, ele despoleta um contentor **Docker** personalizado equipado com **Pan-**

doc, LaTeX e Python. Este contentor executa um script Python que corre consultas, analisa a base de dados usando **Pandas**, gera gráficos estatísticos (PNGs), atualiza um modelo dinâmico em **Markdown** e compila-o via **Pandoc** num relatório final em PDF. O PDF compilado é copiado automaticamente para um volume servido por um servidor web **Caddy**, permitindo aos utilizadores descarregar o relatório mais recente.

- **Competências Chave:** Dockerfiles Customizados (Pandoc + LaTeX + Python), Automação Bash (monitorização de base de dados/pastas), Desenho de Gráficos em Python (Matplotlib/Seaborn), Compilação de PDF com Pandoc, Bases de Dados SQL, Servidor Web Caddy/Nginx.

6. Sistema de Gestão de Energia Doméstica Inteligente com Wiki Técnica

- **Descrição:** Cria um rastreador de energia doméstica inteligente apoiado por documentação técnica interativa. Simuladores publicam métricas de consumo de energia doméstica de hora a hora via **MQTT** para um *broker* **Mosquitto**. Um serviço de recolha **FastAPI** armazena esta telemetria no **PostgreSQL**. O **Grafana** é implementado para exibir tendências de energia e despoletar alertas quando os limites são violados. Para documentar a arquitetura técnica, um contentor **BookStack** ou **DokuWiki** é implementado juntamente com o sistema, pré-populado através de um volume montado com pelo menos 5 páginas markdown detalhando subredes de rede de contentores, esquemas de bases de dados e instruções de configuração. Um **script Bash** corre periodicamente para verificar o uso de CPU e memória de todos os contentores, registando a pegada de recursos.
- **Competências Chave:** MQTT (Mosquitto), PostgreSQL, Grafana, Implementação de Wiki (BookStack/DokuWiki), Volumes e Redes Docker, Telemetria de Recursos com Bash.